

Better Manipulation for Robot Agents

System 0 Actions and Grounded Force–Position Control

Research Proposal I · Zimo Wen

September 2026

Abstract

Robot-RSI commonly expresses a manipulation goal as an end-effector target that an inverse-kinematics and trajectory backend converts into robot motion. This proposal asks when better manipulation requires a different action interface: a fast local policy that directly predicts robot actions, an explicit statement of desired interaction passed to a fixed feedback controller, or a learned local force policy conditioned on that statement. The research is grounded in specific contact mechanics rather than assuming that lower-level output is always superior. Force Policy already predicts interaction frames, force-control masks, reference wrenches, and motion chunks; these are prior capabilities, not proposed inventions. The intended contribution is a task-grounded interface that connects an agent’s interpretation of the scene to executable interaction requirements, with matched comparisons against direct-action and existing force-policy baselines. Success, contact quality, generalization, and cost jointly determine whether manipulation improves.

1. Problem Definition and Evidence from Existing Systems

What System 0 means here. We use System 0 operationally for a fast local sensorimotor policy below task reasoning, not as the name of a unique checkpoint or a claim that every such policy outputs torque. Its output might be joint position, joint velocity, EEF increments, or torque. These interfaces must be distinguished. ACT explicitly predicts future target joint positions [1]; Reactive Diffusion Policy uses a slow latent policy and a fast tactile-conditioned decoder [2]. Neither observation alone establishes arbitrary joint-torque control.

The current baseline. In the inspected Robot-RSI LIBERO path, EEF targets are passed through pose solving and servo execution to joint increments. This is a concrete target-plus-IK baseline. A direct joint-action policy can avoid a separate target-to-joint IK solve at its own output boundary, but it still needs a robot servo, limits, and dynamics. Conversely, predicting an EEF delta can still invoke IK or a Jacobian-based controller internally. Eliminating a named IK call is not itself a manipulation result.

What Force Policy already solves. Fang et al.’s Force Policy [3] predicts an interaction frame, a six-axis selection mask, reference wrench, and motion chunk. Its global vision and local force policies are asynchronously scheduled. The paper also uses task semantics and a VLM in demonstration-based interaction-frame recovery. Therefore, neither adding desired force to an output nor using language to help determine interaction structure can be claimed as new by itself.

A more specific hypothesis. An agent can improve manipulation by selecting and updating *scene-grounded interaction requirements*: which motion is intended, which contact should be maintained, in what frame, and within which task-supported force range. These requirements can drive fixed control or condition a learned local policy. The hypothesis is only supported if this improves over existing Force Policy and appropriately tuned controllers under matched information and data. Otherwise the result may be a useful action-interface comparison rather than a new control method.

Existing integration is not an experiment. A separate Robot-RSI force-control integration documents official ForcePolicy adapters, processors, a scheduler bridge, and hybrid-control streaming. Its recorded validation used fake SDKs and substitutes; it did not establish real checkpoint inference or hardware contact performance. This is more infrastructure than contact-stop alone, but it remains an implementation starting point rather than evidence for the proposed paper.

2. Approach A: Direct System 0 Action Prediction

Input and output. A slow agent supplies an instruction or bounded task-phase representation z_t . A local policy receives z_t , current images or visual features, proprioception, and a short sensor history. The primary direct-action variant predicts joint-position or velocity sequences. A position target can be executed by a fixed servo, illustratively $\tau = K_p(q^{\text{ref}} - q) - K_d\dot{q} + \tau_{\text{ff}}$, where the servo gains and feedforward terms belong to the declared backend. The language model does not generate each servo-cycle torque token.

What is being learned. Train the local mapping from the same physical demonstrations used by the EEF-target comparison, retaining actual commanded joint targets rather than substituting measured positions indiscriminately. A direct policy may learn contact-sensitive corrections and avoid poor target-to-joint choices, but it can also overfit embodiment and workspace geometry. Evaluate both ordinary manipulation and configurations where the existing pose solver struggles. Report solver failures separately from failures of the policy’s intended behavior.

Additional variants. Compare direct joint commands with a local EEF-increment head and, only when a torque interface and training setup are validated, a bounded torque or residual head. The EEF head is not labeled IK-free if its backend performs pose inversion. Keep sensing, action horizon, training data, and servo limits matched. The question is whether changing the executable interface improves success and contact behavior, not whether one interface is syntactically closer to a motor.

3. Approach B: Interaction Requirements Realized by Fixed Control

A grounded interaction contract. The agent emits $c_t = (\mathcal{F}_t, S_t, \mathcal{W}_t^{\text{ref}}, \xi_t^{\text{motion}}, \mathcal{B}_t, \mathcal{G}_t)$: an interaction frame, force/position axis mask, reference wrench, desired motion, calibrated bounds, and phase/termination conditions. The contract records the source of each physical quantity, such as a calibrated tool profile, measured demonstration, visible geometry estimate, or bounded development experiment. A phrase like “press gently” does not uniquely determine a force in newtons.

What the fixed program does. The program converts frames, validates units and signs, selects compatible motion and force axes, and runs a feedback controller. For a simple admittance realization, an illustrative velocity reference in one common frame is

$$\xi^{\text{cmd}} = (I - S)\xi^{\text{motion}} + SA(\mathcal{W}^{\text{ref}} - \mathcal{W}^{\text{meas}}),$$

with dimensioned translational/rotational admittance A . A vendor hybrid controller can instead implement this behavior internally. Robot dynamics, contact, saturation, and the motion backend still matter. There is no unique algebraic inversion from a desired force to a joint configuration; force is achieved through closed-loop interaction.

Physical interpretation. A desired wrench is different from measured wrench, a maximum-force stop condition, and impedance parameters. Express force and moment in a specified frame and at a specified origin; moving the wrench origin requires the appropriate moment transformation. Document whether the SDK represents force felt by the TCP or force exerted on the environment. Stiffness and damping may be fixed controller settings or additional declared outputs, but they are not interchangeable with the force-axis mask used by Force Policy.

Uncertainty. If geometry, force provenance, or the relevant task phase is uncertain, the agent must request an allowed observation or use a declared conservative calibration procedure. It must not invent a target force. All development identification and calibration costs are included when comparing with fixed or manually specified contracts.

4. Approach C: A Contract-Conditioned Local Force Policy

Method. Condition a learned local policy on the grounded contract, wrist observations, and force and proprioceptive history. It updates the interaction frame, mask, reference wrench, and local motion within declared task bounds. The motivation is that the agent specifies the intended physical outcome while the fast policy adapts to local contact. This reuses the established Force Policy representation; the research variable is the value and reliability of the explicit task-grounded conditioning and its updates.

Required comparison. Compare the official Force Policy architecture and labels with an otherwise matched contract-conditioned version, including a hand-specified-contract upper reference and a no-contract ablation. Preserve its processor, frame conventions, and task-specific labeling choices where applicable. Do not attribute gains from extra demonstrations, a different scheduler, or a stronger vision backbone to the contract. The authors report a 50 Hz local policy and unified scheduling; those are source-specific measurements, not achieved frequencies for our integration.

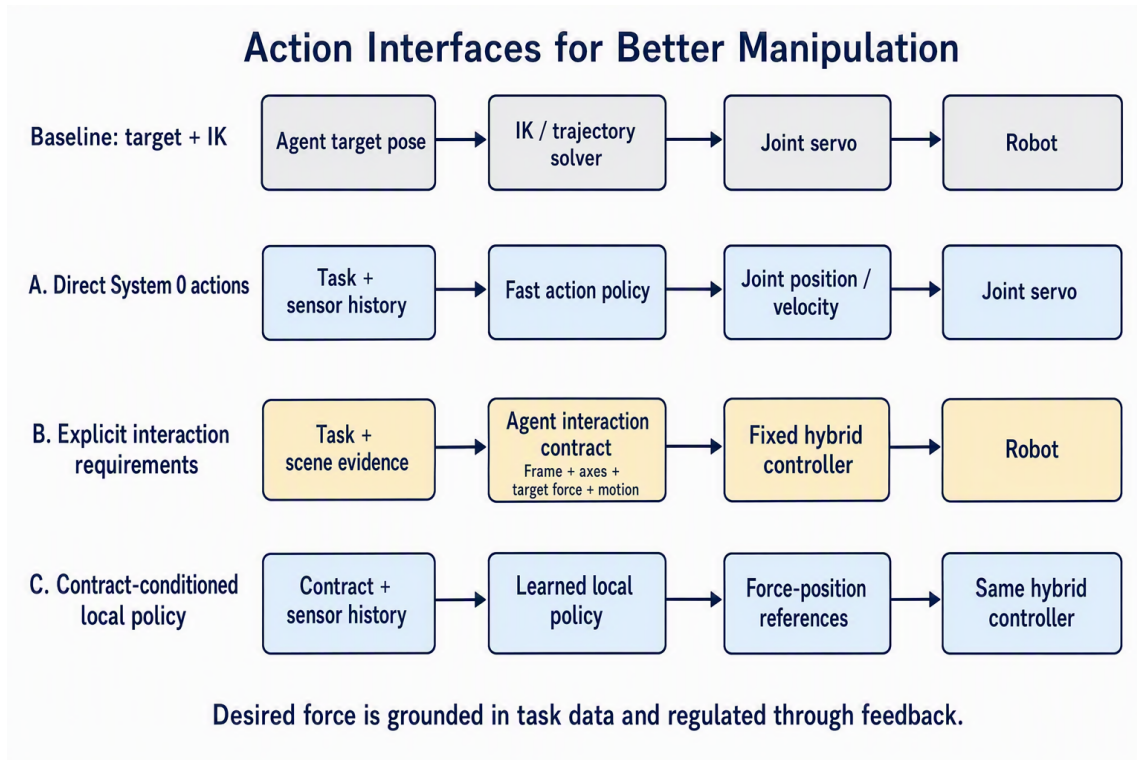


Figure 1: Target-plus-IK is the reference. A predicts direct local actions; B grounds interaction requirements for fixed feedback control; C conditions a learned local policy on those requirements. GPT-image-2 conceptual figure. Each backend and actual command space must be documented.

Why an agent could add value. Test task-phase or instruction changes that alter the desired interaction while preserving object appearance: maintaining contact versus withdrawing, or surface following versus a position-only approach. A contract can make this distinction explicit, but existing learned policies may already infer it. Only matched experiments on held-out task–geometry combinations can establish the agent’s additional contribution.

5. Concrete Scenarios and Experimental Design

Scenario	Appropriate interaction requirements	What must not be assumed
Surface following with a tool or pad	Track tangential motion, regulate normal contact, and specify tool orientation and force provenance.	The same normal force works for different surfaces, materials, or tools.
Constrained insertion	Advance along the identified insertion axis; permit declared lateral/angular compliance and cap axial resistance.	Higher insertion resistance should always be answered by larger constant pushing force.
Drawer or articulated sliding	Follow the estimated motion constraint; use lateral compliance and resistance/position feedback.	Drawer resistance is a universal desired axial-force target rather than a changing observation or limit.

Choosing force references. For surface following, derive an admissible normal-force range from development demonstrations and tool/material calibration, then select or predict a reference within that range. Report the task effect, such as sustained contact or achieved surface coverage, together with material-specific upper limits. For insertion, start with a declared low-speed motion and resistance limit; zero lateral wrench targets or an impedance policy may permit alignment only when the actual controller supports those semantics. Desired force is used only when the task truly requires it. No generic numeric force values are proposed as transferable robot settings.

Initial panel. Use four held-out object/geometry configurations in each of the three task families, three levels of relevant physical variation, and five initial seeds: 180 task episodes per method. Development and validation use distinct objects and task–geometry combinations. Test single changes before combinations: surface tilt, compliance, friction, insertion clearance, or articulated-axis error. Verify that the task remains physically solvable. These contact cells are declared extensions; standard LIBERO scores are reported separately.

Baselines. Include the current target-plus-IK path with tuned feedback; direct joint-action and local EEF-action policies; manually specified fixed hybrid control; the published Force Policy method where data and interfaces support it; agent-generated contracts with fixed control; and contract-conditioned local learning. Reactive Diffusion Policy provides an additional slow–fast sensorimotor comparison where its data requirements can be matched. Record the exact output space of every method instead of classifying all learned actions as torque control.

Data and hardware realism. Use synchronized images, commanded and measured joint/EEF motion, and calibrated wrench history. Force targets and interaction-frame labels need appropriate processing; a force sensor stream is not automatically an optimal reference trajectory. Force Policy’s public Push and Flip checkpoint is not a model for arbitrary insertion, scraping, or cutting. Obtain task-matched training data for comparisons. Hardware evaluation requires validated tools, frames, sensing, and control interfaces; no robot execution is part of preparing this proposal.

Standard task reporting. On the separate 120-task LIBERO short panel, report frozen task-level pass@10 with spatial/object/goal and task/object/swap/lan breakdowns. Keep original benchmarks, contact extensions, historical adaptive results, and trained-policy results distinct.

6. Planned Ablations and Evidence for Better Manipulation

Experiment A: action space versus controller quality. Compare target-plus-IK, direct joint actions, and local EEf actions using the same demonstrations, sensing, and comparable model capacity. Tune the IK baseline and disclose the trajectory and servo settings. Evaluate task completion, contact disturbances, and motion quality, including held-out configurations. A direct policy beating an untuned solver does not establish a general action-representation advantage.

Experiment B: desired contact versus force limits. Compare position control with force sensing, contact-stop behavior, fixed hybrid references, and task-grounded references. All methods observe the same wrench signal. Separately ablate frame estimation, axis selection, desired wrench, and motion intent. This distinguishes the value of force feedback from the value of an explicit target. A clamp that prevents large forces is not evidence that the intended force was tracked.

Experiment C: what the agent contributes beyond Force Policy. Compare no contract, fixed human-specified contracts, generated contracts, and generated contracts updated after observable contact changes. Match training labels, local network, and scheduler. Test instruction/phase changes and new task–geometry combinations while holding the scene appearance approximately constant where possible. Add incorrect-frame, incorrect-mask, and unsupported-reference diagnostics only offline in simulation and label them clearly. If existing Force Policy matches performance, limit the novelty claim to the measured task interface or generalization setting.

Experiment D: grounding and calibration. Compare references obtained from task-matched demonstrations, calibrated ranges, and language-only numeric proposals as a simulation diagnostic. The last condition tests the cost of ungrounded specification, not an endorsed deployment strategy. Measure whether the proposed contract predicts a useful physical quantity and whether its provenance supports transfer to a new object. Calibration trials and extra sensing are counted as resources.

Metrics. Final simulator/harness success is primary. During defined contact phases, measure reference-force RMSE, contact-loss duration, peak force, impulse, and task-specific physical effects. Report pose error and settling behavior in motion phases. Force tracking alone can be perfect for a useless reference, so it must be paired with actual manipulation success. An insertion that reaches high force without insertion progress is a failure, not successful force regulation.

Observed pattern	Supported interpretation
Direct actions help pose-solver-sensitive cases only	A useful embodiment/interface boundary, not universal superiority of lower-level control.
Grounded references improve contact and task success	Supports task-specific interaction specification, after matching feedback access.
Force tracking improves but task success does not	The chosen physical objective may be inappropriate.
Contract conditioning matches unmodified Force Policy	No incremental learned-control contribution has been established.
Manual contracts work but generated ones fail	Grounding the agent’s physical specification is the bottleneck.

Use paired seeds and task-clustered intervals, with multiple training seeds for learned variants. Report inference, calibration, training, and execution costs separately. Additional samples are determined from validation variance; the proposed panel is not an assertion of sufficient statistical power.

7. Implementation Boundaries and Expected Contribution

Reuse before invention. The inspected Force Policy code and paper already expose action/frame/wrench/mask outputs, task-specific processors, and a slow-fast scheduler. The Robot-RSI integration documents frame conversion and fixed SDK realization. Reuse these interfaces as declared baselines rather than rebrand them. The new paper must identify a measured gain from task-grounded specification, direct local action learning, or their combination; these are hypotheses, not established capabilities.

Control facts that must remain explicit. Joint targets still require a servo. EEF targets may still require IK. A desired wrench needs a contact model or feedback realization, not a unique inverse solution. Impedance sets a force-motion relationship, while hybrid control selects which axes regulate motion or force. Wrench frame, origin, sign, units, and sensor compensation are part of the method. SDK command acceptance, completed streaming duration, or a fake-interface test does not establish contact convergence or task success.

Scope of the agent and the local policy. The agent selects task phases and grounds interaction requirements on a slower timescale. The local policy or fixed feedback program performs sensor-conditioned execution. Actual update rates and delays must be measured. We do not claim that GPT emits reliable high-frequency torques. System 0 is the functional placement of the local sensorimotor module, whose action representation and training are specified separately for each variant.

Evidence and failure accounting. Final outcomes come only from the simulator or independent harness. Exclude provider, transport, image, resource, and interrupted records from valid-verdict success denominators and report them separately, with implementation errors and coverage. Retain force curves, videos, rejected specifications, failed runs, and exact manifests. Report tokens, VLM and unmetered calls, wall time, and perception/action/recovery time. Learned models require matched and held-out simulation evaluation; training loss does not establish manipulation improvement.

Expected paper contribution. The strongest plausible result is an empirically supported interface between task reasoning and physical interaction: the agent specifies grounded, scene-appropriate motion and contact requirements, and an explicitly defined local realization executes them better under selected generalization conditions. A useful negative result is that direct actions or published Force Policy already suffice. Either outcome must explain concrete manipulation behavior rather than equating lower abstraction with greater intelligence. The second proposal separately studies generated reactive policy structure; the third retains GPT to audit and correct VLA actions.

References

- [1] Zhao et al. Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ACT). 2023. <https://arxiv.org/abs/2304.13705>
- [2] Xue et al. Reactive Diffusion Policy: Slow-Fast Visual-Tactile Policy Learning for Contact-Rich Manipulation. 2025. <https://arxiv.org/abs/2503.02881>
- [3] Fang et al. Force Policy: Learning Hybrid Force-Position Control Policy under Interaction Frame for Contact-Rich Manipulation. 2026. <https://arxiv.org/abs/2602.22088>
- [4] Force Policy official source, model heads, processors, and scheduler. <https://github.com/force-policy/ForcePolicy>
- [5] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [6] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

Robot Agents That Generate Reactive Policies

State Machines, Stateful Programs, and Constraint-Based Execution

Research Proposal II · Zimo Wen

September 2026

Abstract

A robot agent may write an executable plan that becomes obsolete while the robot is acting. Repeated language-model calls can update the plan, but they place inference latency inside the response path. A manually written collection of if–else rules avoids that delay without demonstrating an agent-specific research contribution. This proposal instead asks whether an agent can synthesize a task-appropriate reactive policy whose local execution continuously incorporates new observations. Three representations are compared: a generated state machine, a stateful feedback program, and a generated objective/constraint specification solved locally. All use the same sensors and motion backend. The paper tests whether the generated feedback structure transfers to unseen tasks and changes, reduces unnecessary replanning, and improves actual recovery at a matched computation budget. State machines are one candidate representation within a broader policy-synthesis problem, rather than the full definition of responsiveness.

1. Research Question: Generate a Policy, Not Only an Action List

A motivating difference. An open-loop plan says “move to location A, descend, close the gripper.” A reactive policy says “track the current cup estimate, descend when alignment is sufficient, and close only when the grasp condition is observed.” If the cup moves slightly, local execution updates its target without waiting for another complete agent turn. If the cup disappears or the task changes, the policy identifies an invalid assumption and escalates. The distinction concerns the dependence of actions on observations, not the number of if–else statements.

Agent-specific contribution. A fast controller or a background thread can be useful without being a new Robot Agent method. Here the agent must infer what needs monitoring, which observations change the current behavior, what memory is required, and when its local policy stops being applicable. The generated object is a reusable feedback mapping, not simply a fixed sequence with a more frequent timer. Its value is evaluated on task and disturbance combinations not encoded by a developer in advance.

Problem formulation. A synthesis model maps instruction g , initial permitted observations, and a declared primitive library to a policy P . At local execution step t , P maps observation o_t and memory z_t to an action reference, updated memory, and an explicit status: continue, complete, or escalate. The low-level motion realization is fixed across comparisons. This separates policy representation from the first proposal’s physical action interfaces and from the third proposal’s GPT audit of a VLA candidate.

Existing work. Code as Policies already generates programs that may include feedback. Real-Time Chunking addresses asynchronous execution of learned action chunks; Reactive Diffusion Policy and Force Policy provide slow–fast contact architectures. We cannot claim novelty for feedback loops or asynchronous execution alone. The proposed question is whether *task-conditioned synthesis of feedback structure and applicability conditions* produces better generalization and response than open-loop programs, fixed generic monitors, and developer-written state machines.

Hypotheses. H1: generated task-specific feedback improves disturbed-task completion over fixed sequences at matched sensing and action access. H2: state machines, stateful programs, and constraint policies have different strengths across discrete mode changes, continuous tracking, and geometric replanning. H3: explicit validity and escalation conditions reduce costly full replanning while preserving recovery. If a generic handcrafted monitor performs equally well, the agent-specific claim must be narrowed.

2. Approach A: Agent-Generated State Machines

Policy representation. Represent a policy as states, state-local actions, observable guards, and transitions. For grasping, possible states are observe, approach, align, grasp, lift, and recover. The agent chooses this structure from the task, rather than merely filling in a fixed task-specific graph. A transition might require stable alignment for several observations; recovery might depend on an observed grasp inconsistency. Store timers and short measurement histories where necessary to avoid switching on one noisy frame.

Generation contract. Each guard names its observation source, freshness requirement, threshold units, and action consequence. States specify entry behavior, local update, timeout, and exit. The runtime checks missing transitions, unreachable states, loops without bounded progress, incompatible action ownership, and absent observations. These checks establish structural executability, not task success. Guards cannot call a hidden simulator success predicate or use exact target geometry unavailable through sensors.

Local execution and repair. The state machine consumes a stream of code-backed perception and proprioception at a fixed measured rate. It issues short-horizon references through the common motion backend. An invalid assumption triggers a bounded agent revision of the relevant state or transition; unaffected states are retained. Test-time revision remains within the current episode and is logged separately from development-time skill accumulation. Compare generated graphs with fixed generic graphs and task-specific human graphs.

3. Approach B: General Stateful Feedback Programs

Beyond discrete branches. Some behavior is naturally continuous or depends on history. A program can estimate target velocity, smooth noisy observations, maintain object identity through brief occlusion, and continuously update an approach offset. It need not enumerate every possible target location as a separate state. The agent generates a local function with the common contract

$$(u_t, z_{t+1}, r_t) = P(o_t, z_t), \quad r_t \in \{\text{continue, complete, escalate}\}.$$

Here u_t is a motion reference and z_t stores permitted estimates and timers. The discrete status describes execution management; the action mapping itself can be continuous and stateful.

Example. A tracking policy computes the current grasp approach from the latest observed object pose and a task-specific offset, updates a filtered velocity estimate, and reduces progress when uncertainty rises. It transitions toward grasping only when measured alignment and observation freshness satisfy the policy. Persistent loss of the target produces escalation. The robot does not chase a coordinate frozen at program-generation time.

Implementation boundary. Programs use declared geometry, filtering, perception, and motion functions in a restricted runtime. No arbitrary filesystem access, simulator introspection, or unbounded remote model calls occur inside the local step. Bound memory, step duration, output magnitude, and revision count. If the backend only offers a blocking full-trajectory call, expose a validated short-step interface before claiming continuous response; adding a Python loop around blocking calls does not establish the required behavior.

Comparisons. Remove memory, freeze target updates, and replace task-conditioned logic with a generic tracker while preserving the same data stream. Compare full regeneration with local patching. If performance comes entirely from a stronger perception module or more frequent observations, the program-synthesis contribution has not been isolated.

4. Approach C: Generated Objectives and Constraints

Representation. The agent generates a task-specific cost and observable constraints, then a fixed local solver updates actions as the scene changes. For example, reaching toward a moving target can balance current goal error and motion smoothness while respecting estimated obstacle clearance. A short-horizon controller repeatedly solves

$$\min_{u_{t:t+H}} C_P(\hat{s}_t, u_{t:t+H}) \quad \text{subject to} \quad g_P(\hat{s}_t, u_{t:t+H}) \leq 0,$$

where $\hat{s}_t$ is estimated from permitted observations. The agent determines the task-dependent objectives, constraint sources, and applicability conditions; the solver supplies numerical optimization. No exact hidden obstacle pose enters the constraints.

Why this is a broader policy. New target and obstacle estimates change the optimization directly, without requiring a new if-else branch for every layout. Discrete stage changes can be represented by a small supervisor. Infeasibility, stale observations, or an invalid local dynamics approximation trigger escalation. A complex solver is not assumed superior; compare simple analytic tracking and a fixed generic objective under matched runtime budgets.

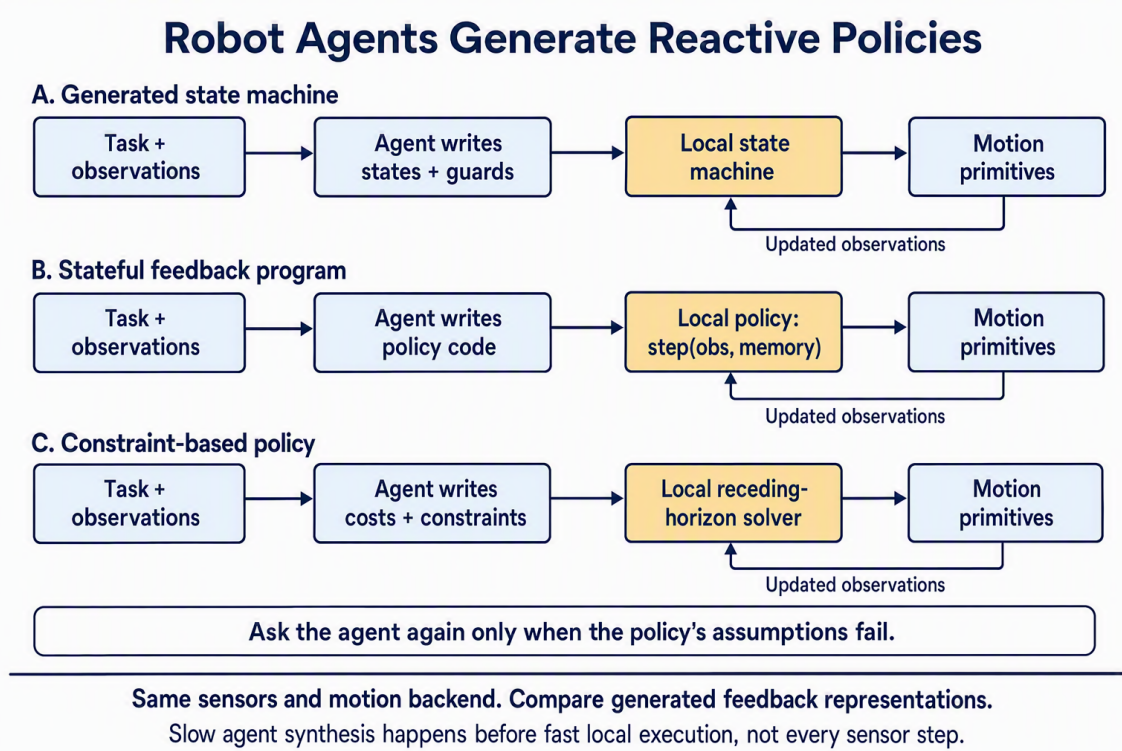


Figure 1: The agent synthesizes one of three feedback representations; updated observations enter its local execution loop. The sensor stream does not require a new language-model call at every step. GPT-image-2 conceptual diagram. All methods share perception and motion primitives.

Common policy lifecycle. Synthesize, validate, initialize memory, execute local updates, check completion or applicability, and request a bounded revision if needed. Record the generated policy version and the observation that justified each revision. Replacing a policy requires a controlled handoff; an obsolete asynchronous result cannot overwrite a newer policy. This lifecycle is shared infrastructure, not three separate higher-performance controllers.

5. Experimental Design: Generalization of Reactive Structure

Task and disturbance families. Use LIBERO short tasks with matched disturbances: target movement during approach, transient occlusion, changed visible path clearance, grasp inconsistency, and instruction or stage changes. Disturbances remain physically meaningful and leave the task solvable. The test set includes unperturbed tasks to measure regressions caused by unnecessary feedback and escalation. Hidden event schedules stay with the evaluator.

Task-level splitting. Group near-duplicate recipes and their language/layout variants before creating development, validation, and test splits. The synthesis model sees the test instruction and initial observations, but does not receive hand-coded rules for its event sequence. Hold out both tasks and task–disturbance combinations. Report whether the primitive library already contains a full task solution; otherwise retrieval of that skill could masquerade as new reactive synthesis.

Proposed panel. Begin with 24 held-out task–disturbance pairs, three strengths, and five seeds: 360 episodes per method. An additional matched subset varies observation or model latency. Determine the actual delay levels from validation relative to the task timescale, rather than copying arbitrary millisecond targets. Task pairs sharing a recipe remain one statistical cluster. The panel size is provisional and must be assessed against development variance.

Comparison	What is generated or fixed	Question isolated
Open-loop program	Fixed action sequence	Value of observation dependence
Repeated agent replanning	New plan at fixed intervals	Response benefit versus inference cost
Generic manual monitor	Fixed event rules and recovery	Value of task-specific synthesis
Task-specific manual FSM	Human-designed states and guards	Strong structural reference
Generated FSM	States, guards, and transitions	Discrete feedback synthesis
Stateful policy program	Continuous actions and memory	Broader feedback representation
Constraint-based policy	Task costs and constraints	Value of local optimization

Fairness. All methods use identical perception outputs, timestamps, short-horizon motion commands, and motion-controller limits. Match local execution rates and separately compare fixed total VLM-call or token budgets. A manual state machine may require more human design effort, which should be documented separately rather than converted into an arbitrary performance penalty. Do not give generated policies additional hidden sensing or stronger task primitives.

Two evaluations. A policy-level diagnostic injects observations into frozen generated policies to inspect guard decisions, memory behavior, and output validity. It does not count as robot task success. Full closed-loop simulation then measures execution under actual changing observations. The standard 120-task LIBERO panel separately reports frozen task-level pass@10 by spatial/object/goal and task/object/swap/lan, without mixing dynamic-extension results into the headline.

6. Planned Ablations and Meaningful Response Metrics

Experiment A: generated feedback versus more rules. Compare generic manual rules, task-specific manual FSMs, and generated policies with a fixed primitive set. Hold code size or synthesis-token budget approximately controlled in a secondary panel, but do not use the number of branches as a capability metric. Test whether generated guards monitor the right task-dependent quantities and whether the resulting correction helps after the event. More conditions can increase false triggers rather than improve response.

Experiment B: discrete, stateful, and optimization-based policies. Evaluate mode changes, continuous target drift, brief occlusion, and changing obstacle geometry separately. Expected strengths are hypotheses: FSMs may suit discrete stages, stateful programs may preserve tracking through ambiguous observations, and constraints may help spatial adjustment. Test these claims rather than selecting only tasks favorable to each representation. Report when a simpler representation achieves equal success at lower synthesis or runtime cost.

Experiment C: memory and applicability. Remove history, smoothing, freshness checks, and escalation conditions separately. Compare local patching with regeneration of the entire policy. Measure whether the agent recognizes an invalid assumption and revises the responsible component, or repeatedly changes unrelated code. Test-time policy versions and their triggers are retained. Failed patches and excessive escalations count against task and resource budgets.

Experiment D: response under delay. Measure the event time, first observation containing it, local policy update, controller acceptance, and actual corrective actuation. Local policies should absorb changes within their declared validity domain without waiting for an agent call. Out-of-domain events require slower reasoning and must be reported separately. Compare with asynchronous action-chunk execution where appropriate, but do not attribute a scheduler’s existing advantage to policy synthesis.

Primary metrics. Report final disturbed-task success, harmful response on unperturbed tasks, false/missed triggers, intervention count, escalation rate, response p50/p95, recovery time, and VLM cost. A program detecting the event but failing to change the robot is not responsive. A program that stops instantly but never resumes has a stop-path benefit, not successful manipulation recovery. Nonresponses remain visible with their timeout rate and censoring, rather than disappearing from latency summaries.

Possible result	Supported conclusion
Generated policies generalize and reduce replanning cost	Supports synthesizing task-dependent feedback structure.
Manual generic rules match performance	The agent-specific contribution is limited in this task set.
Stateful programs help only during occlusion	Evidence for memory in a specific observable regime.
Constraint policies reduce geometric failures but cost more time	A representation–compute tradeoff, not uniform superiority.
More code increases false escalation and regressions	Feedback complexity requires validity and relevance checks.

Use matched seeds, task-clustered confidence intervals, and multiple synthesis samples where generation is stochastic. Freeze prompts and selection rules before testing. Select checkpoints or programs using validation only; success-conditioned cherry-picking of generated code cannot establish deployment reliability.

7. Feasibility, Risks, and Expected Paper Contribution

Robot-RSI integration. The Planner or Engineer generates the policy representation, the runtime validates and executes it over existing perception and motion primitives, and visible traces support bounded revision. The proposal does not require a newly learned force controller. Existing implementations have different cancellation and streaming capabilities, so document the selected backend and hold it fixed across methods. A lock fix or faster transport alone is infrastructure work, not the policy-synthesis contribution.

Limits of verification. A syntactically valid state machine can contain incorrect guards; a terminating program can use an inappropriate recovery; an optimizer can satisfy a badly specified objective. Structural checks must be complemented by actual simulator outcomes. Robot observations are imperfect, so policies need a stated uncertainty/freshness domain. Neither handcrafted guards nor agent-generated constraints are presumed to guarantee safety or task completion.

Preventing hidden-state leakage. The agent, local code, and reviewers receive only declared images, estimated geometry, proprioception, and tool receipts. Hidden task predicates, future events, object ground truth, and private evaluator checkpoints are inaccessible. Learning a guard from a visible signal is allowed; reading the simulator’s answer to decide a transition is not. Post-hoc adjudication remains independent of the generated program’s own completion flag.

Evidence and cost. Preserve generated policies, all revisions, videos, trajectories, failures, and exact manifests. Count success only from final simulator/harness verdicts, with provider, transport, image, resource, and interrupted records reported separately outside valid-verdict denominators. Also show implementation errors, invalid-program rates, and incomplete coverage. Record synthesis and revision tokens, VLM/unmetered calls, wall time, perception/action/recovery time, local-step latency, and solver cost. Efficiency claims require task success to remain acceptable, not merely fewer language-model calls.

Expected contribution. The paper should establish whether a robot agent can turn task understanding into a feedback policy that continues to work as observations change. Its methods compare a finite-state representation with more general stateful and constraint-based policies; its experiments isolate generation from stronger sensing and control. The central result is generalization of the reactive structure under a fixed execution backend. This is different from the first paper’s action/force representation and the third paper’s retained GPT audit of VLA proposals.

Scope of the claims. Feedback programs, hybrid systems, local optimization, and asynchronous policies already exist. A new claim must concern how the agent synthesizes, grounds, and revises them, and what measurable benefit this yields. If fixed manual policies suffice, report that outcome. The proposal contains experimental designs, not newly measured success rates or promises of hard real-time execution.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [3] Reactive Diffusion Policy: Slow-Fast Visual-Tactile Policy Learning for Contact-Rich Manipulation. 2025. <https://arxiv.org/abs/2503.02881>
- [4] Force Policy: Learning Hybrid Force-Position Control Policy under Interaction Frame for Contact-Rich Manipulation. 2026. <https://arxiv.org/abs/2602.22088>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>

GPT-6 Orchestration over ROS 2

Task Planning, Code Repair, and Single-Step Arm Correction

Research Proposal III · Zimo Wen

September 2026

Abstract

This proposal studies a robot system in which GPT-6 coordinates the complete task, generates and revises executable control code, and uses execution feedback to decide when the robot needs a local correction. ROS 2 connects observation, action execution, and the existing low-level controllers. The robot can continue a valid local policy without making a language-model request on every control tick. When a task assumption fails, GPT-6 receives a bounded observation trace, corrects the relevant code or action reference, and submits a versioned replacement for execution. The research question is whether selective, feedback-grounded repair improves task completion over one-shot generated ROS programs and full replanning, at a matched sensing, action, and inference budget. This is a proposed architecture and experimental protocol; no ROS integration or performance result is claimed here.

1. Scope and Relationship to the Three Proposals

The requested division of work. Proposal I studies bottom-level manipulation: direct local actions, grounded force–position requirements, and learned contact policies. Proposal II studies immediate response: agent-generated state machines, stateful feedback programs, and locally solved constraints. Proposal III studies **GPT-6 task orchestration and code repair connected to a ROS execution system**. In-context learning is not a standalone proposal in this collection.

GPT-6 stays in the deployed system. GPT-6 plans task stages, produces code against a declared ROS-facing API, interprets allowed execution feedback, and makes the final model-authored code correction. The revised program then passes deterministic interface checks before ROS executes it. These checks do not replace GPT-6’s reasoning, and GPT-6’s approval does not establish physical success. The exact GPT-6 deployment, prompts, reasoning settings, and budgets are frozen and recorded for each evaluation.

ROS has an explicit role. ROS 2 transports timestamped observations and action feedback, manages the execution lifecycle, and connects to robot controllers through a declared driver. A controller such as a servo, trajectory controller, or supported force controller realizes the commands. ROS itself is not a learned policy and does not make a task plan automatically. The generated program must state what it observes, which interface it commands, and what feedback changes its behavior.

Meaning of a single correction. The default study uses a bounded correction to the next end-effector reference or a short declared action segment. Translation, orientation, frame, duration, and gripper behavior are explicit. A joint-space interface can instead support a bounded joint increment, but the two command spaces are separately specified and evaluated. “Single-step” does not mean that a remote model participates in every servo cycle.

Nominal behavior. A nominal action may come from an existing ROS skill, a generated program, or an optional frozen VLA. A VLA is not required by this proposal. The earlier GPT-reviewed-VLA draft is retained as a narrower historical variant; this revision makes whole-task orchestration, code repair, and the ROS boundary explicit.

2. Architecture and Observable Execution Contracts

Observation gateway. A ROS-facing adapter assembles a coherent observation from approved camera streams, measured joint state, gripper state, and available force sensing. Each observation carries acquisition time, frame identifiers, and calibration version. Estimated object geometry is labeled as an estimate. GPT-6 receives only the permitted subset; simulator state, success predicates, hidden object poses, and evaluator-only event schedules are excluded, including debug topics that expose them.

Task supervisor. GPT-6 converts the instruction and current observation into task stages with observable prerequisites, termination conditions, and escalation conditions. The supervisor maintains task identity, current stage, policy version, and the most recent accepted observation. A failed grasp can require local adjustment; a changed user instruction can require a new stage plan. The system logs these as different interventions rather than calling every intervention a correction.

Generated code. The initial executable is a ROS-facing task node or a restricted feedback-policy module hosted by a fixed ROS node. It uses a narrow API for observation, short motion references, gripper commands, cancellation, and feedback. It does not modify the driver or the real-time controller implementation during a live control cycle. GPT-6 may patch a local policy function or stage transition, but must name the old version, the evidence for the edit, and the intended behavioral difference.

ROS execution layer. A practical implementation can use ROS 2 Actions for goal, feedback, result, and cancellation; `tf2` for calibrated frame transforms; and `ros2_control` or a vendor-supported controller for command execution. MoveIt Servo is a candidate for compatible local motion interfaces. Each component must be validated for the chosen arm. Action cancellation being accepted does not imply that physical motion has already stopped; the executor waits for the declared controller acknowledgement and measured state before handing over ownership.

State-valid repair. A repair record includes

$$r = (v_{\text{base}}, t_{\text{obs}}, \mathcal{F}, \Delta u, H, \mathcal{P}, \mathcal{T}),$$

where v_{base} is the program/action version being amended, t_{obs} is the observation timestamp, $\mathcal{F}$ is the coordinate frame, Δu is the bounded edit, H is its validity horizon, $\mathcal{P}$ contains observable preconditions, and $\mathcal{T}$ describes termination and handoff. A result based on an obsolete version cannot overwrite a newer action. If freshness or preconditions fail, execution requests a new observation instead of applying the stale edit.

Control ownership. Exactly one executor owns each arm’s active motion reference. The local controller continues at its measured supported rate; GPT-6 requests occur asynchronously at stage boundaries or escalation events. During a pending model call, a declared locally valid policy or hold behavior remains responsible for the arm. Threading alone is not evidence of responsiveness: timestamps must show when a changed observation actually changes a physical command.

3. Three Repair Mechanisms within the Same System

A: parameter and single-step correction. GPT-6 preserves the program and adjusts a declared next-step reference, such as an approach offset, an orientation increment, or a calibrated controller parameter. For an EEf edit, composition follows a specified tool or world frame rather than adding untyped pose arrays. Joint increments form a separately labeled variant. This establishes whether a small explicit correction is sufficient before crediting unrestricted program synthesis.

B: bounded code patch. GPT-6 changes a policy function or a stage transition while retaining unaffected code. For example, replace a fixed approach target with a fresh observed target and a bounded alignment loop. A patch declares the input observations, output command space, applicable stage, and maximum execution duration. Syntax, supported calls, frame/unit consistency, and resource bounds are checked before installation. Invalid patches consume the declared repair budget and remain in the logs.

C: supervisor-assisted stage repair. If the local policy’s assumptions no longer hold, GPT-6 revises the stage plan and generates its replacement ROS-facing program. It then performs a final code review using the same fixed model configuration and the current permitted evidence. The second call is not independent verification: an ablation must establish whether review reduces harmful edits enough to justify its cost. This broader repair is reported separately from a single-step correction.

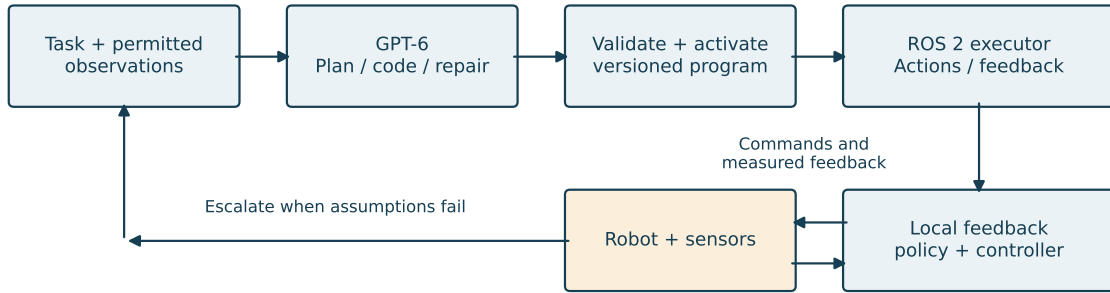


Figure 1: GPT-6 provides whole-task orchestration and code repair. ROS executes a versioned program and routes current feedback to the local policy and supervisor. Fast control remains inside the ROS/controller loop. The diagram describes the proposed mechanism, not an implemented benchmark result.

Common lifecycle. Plan, generate, validate, execute, observe, and either continue or repair. Final model-authored correction precedes interface validation and controlled activation. The previous program and rejected candidates remain available for diagnosis. A code rollback is permitted only when its assumptions still match the current physical state; restoring old source code does not restore the world.

4. Experimental Design and Matched Comparisons

Implementation sequence. First build an observation/action adapter and validate its semantics independently of GPT-6: frames, command scaling, gripper state, clock handling, and cancellation acknowledgement. Next connect generated task programs and single-step corrections. Finally enable bounded code patching and stage repair. Existing LIBERO and RoboTwin environments can support separate ROS-adapter research runs; their current non-ROS runs are not evidence for this architecture. Real-arm evaluation requires a validated driver and task-specific operating envelope.

Task families. Include grasp-place with changed object position, placement alignment, interrupted transport, and compatible articulated-object operations. Add contact tasks only where the sensing and controller semantics have been established. Dynamic conditions include target displacement, brief observation loss, and an instruction change. The evaluator controls disturbance schedules privately and verifies that tasks remain physically meaningful.

Proposed diagnostic panel. Start with 12 task families, four conditions per family (one unperturbed and three declared disturbances), and five seeds: 240 episodes per method. This is an experimental design, not a completed dataset. Development tasks determine disturbance magnitudes and repair limits. Hold out task-condition combinations for evaluation, group related recipes together, and report whether a task’s nominal skill was already in the library. Use the same starting states and disturbance schedules across methods where possible.

Comparison	Question isolated
Fixed ROS task policy	What does the existing executable system achieve without model-generated repair?
GPT-6 one-shot ROS program	Does initial program generation suffice?
GPT-6 full replanning	Does local repair improve on regenerating the program after every escalation?
GPT-6 parameter repair only	Are small action corrections sufficient?
GPT-6 bounded code repair	Does changing feedback logic add value beyond parameter edits?
Full supervisor with final review	Does orchestration plus reviewed repair justify its extra inference?

Fairness. Keep the ROS driver, sensing, motion primitives, controller limits, model snapshot, and nominal skill library fixed. Match action and model-call budgets in the primary comparison; separately report actual incurred costs. If an optional VLA is used, freeze it across the compared methods and identify that experiment as a VLA-backed variant. A direct-SDK adapter comparison can estimate ROS transport overhead under the same control semantics; it does not by itself evaluate GPT-6’s reasoning.

Standard task reporting. Keep the custom ROS/disturbance panel separate from standard LIBERO and RoboTwin results. Any standard LIBERO short leaderboard uses the declared 120-task pass@10 protocol with subset and suite breakdowns. Do not merge the custom panel, single-step diagnostics, and standard benchmark episodes into one favorable success figure.

5. Measurements, Ablations, and Result Interpretation

Primary outcomes. Report final simulator/harness task success, completion time, and useful versus harmful interventions. A compiled program, an accepted ROS Action, or GPT-6’s approval is not task success. Infrastructure, transport, missing-package, and interrupted attempts are separately categorized and excluded from task denominators. Preserve failed programs, ROS observations, action traces, and exact launch/model manifests.

Responsiveness. Measure event-to-observation, observation-to-detection, detection-to-model-request, model turnaround, patch validation, cancellation acknowledgement, and first effective corrected command. Report median and tail latency and the number of missed validity windows. An experiment that pauses simulation while GPT-6 reasons is labeled separately; it cannot establish responsiveness in a changing physical scene.

Repair quality. Measure patch acceptance, execution errors, local recovery, subsequent task success, intervention magnitude, and harm to actions that would otherwise succeed. In a simulator-only diagnostic, evaluate the original and corrected continuation from an evaluator-owned checkpoint. Lock the correction decision before the hidden paired outcomes are computed. The branch outcomes remain post-hoc labels and are never returned to GPT-6 for online selection.

Efficiency. Record prompt, completion, and total tokens; model and perception calls; unmetered calls; wall time; and perception, action, recovery, and review time. Measure the cost of a generated program that keeps working without another model call. A smaller token count caused by premature failure is not an efficiency improvement. Require success and intervention harm to remain acceptable when comparing cheaper variants.

Ablation A: what does GPT-6 contribute? Compare one-shot generation, full regeneration, bounded patches, and parameter edits with identical primitives. Remove the final GPT-6 code-review call while matching total inference budget in one experiment. If a fixed local monitor with parameter updates performs equally well, the evidence supports that simpler mechanism rather than a broad code-repair claim.

Ablation B: is ROS responsible for the gain? Hold the observation and command interface fixed while changing the adapter, where a direct SDK implementation is available. Separate middleware latency, action-management correctness, and model decision quality. Replacing an incomplete blocking driver with a better driver is an engineering improvement, but must not be attributed to GPT-6-generated reasoning.

Ablation C: state freshness and correction granularity. Compare version-checked activation with a separately labeled stale-edit diagnostic, single-step edits with short-segment edits, and parameter-only with code-level changes. Test latency and observation noise at development-calibrated levels. Report both the range in which local repair helps and the conditions in which the task requires broader replanning.

Possible outcomes. Improved completion with fewer full replans supports selective repair. Equal performance from one-shot programs weakens the repair hypothesis. A review call that increases latency without reducing harmful edits is not justified. Gains only under paused simulation identify a latency limitation. Better local alignment without better final success is insufficient evidence for the proposed system.

6. Contribution, Boundaries, and Reproducibility

Intended contribution. The research contribution is a measured way to connect GPT-6 task reasoning and code repair to state-valid robot execution: explicit repair scope, observable applicability, program/action version ownership, and controlled ROS handoff. ROS integration alone, ordinary feedback loops, and generic code generation are established ideas. Novelty must be demonstrated against matched implementations of those components rather than asserted from their combination.

Distinct scope of the three papers. The first paper changes what the local policy commands and how contact requirements are represented. The second changes how generated local behavior responds to new observations without waiting for another model call. This third paper changes how GPT-6 coordinates those behaviors, decides that they need repair, and installs corrected code through a ROS system. Shared infrastructure may support all three; each paper needs an experiment that isolates its own variable.

Model configuration. Record the actual GPT-6 deployment identifier, supported input modalities, prompts, sampling and reasoning settings, maximum repair rounds, and tool budget. Structured perception is sufficient for a text interface; raw images are supplied only to a deployment that supports them. The name GPT-6 is not a latency, accuracy, or control-rate guarantee. Model services and local controllers have different timing responsibilities.

Physical and statistical limits. Single-step EEF or joint corrections do not establish torque-control ability. Force references require calibrated units, frames, sensing, and a controller with the relevant semantics. Simulation gains do not establish real-arm contact performance. Report paired task-level intervals and cluster related task variants; multiple frames or checkpoints from one episode are not independent successes. Test-set patches are confined to the current episode and do not update the frozen skill library.

Artifacts to retain. Release the ROS message/action contracts, adapter versions, approved observation allowlist, generated programs and patches, model configuration, controller configuration, and evaluation manifest. Separate development corrections from frozen evaluation. Keep rejected and interrupted runs visible. Hidden ground truth remains with an independent evaluator and outside GPT-6, generated programs, and reviewer-visible context.

Status of this document. This revision replaces the third proposal’s VLA-specific emphasis with the requested GPT-6–ROS orchestration and code-repair system, while retaining VLA-backed actions as an optional variant. The first two proposals remain bottom-level manipulation and reactive-policy synthesis. This document does not claim that ROS has been integrated into the currently running remote experiments, and it introduces no measured success rate.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. <https://arxiv.org/abs/2209.07753>
- [2] ROS 2 documentation: Actions, Quality of Service, and tf2. <https://docs.ros.org/en/jazzy/>
- [3] ros2_control: controller and hardware interface documentation. <https://control.ros.org/jazzy/>
- [4] MoveIt Servo: real-time motion-command interface documentation. https://moveit.picknik.ai/main/doc/examples/realtime_servo/realtime_servo_tutorial.html
- [5] LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. <https://arxiv.org/abs/2306.03310>